# Parking Lot System

**Language:** Any

## Problem

A parking lot is a **2D grid**. The map of the lot is given in the input file:

| Symbol | Meaning |
|---|---|
| `E` | Entrance (exactly one) |
| `.` | Road |
| `#` | Wall |
| `P` | Parking slot |

Cars enter at `E` and drive only on road cells. They can move **up, down, left or right**, not diagonally. A car can park in a slot that is directly next to a road cell (or the entrance) that it can reach. A slot that no car can reach is never used.

**Distance of a slot** = the number of road steps from the entrance to the cell next to the slot, plus 1 step into the slot.

When a car arrives, park it in the **nearest free slot**, meaning the one with the smallest distance. If two slots have the same distance, choose the one with the smaller row, then the smaller column. When a car leaves, its slot becomes free again.

Rows and columns start at 1. Row 1 is the top of the map. A slot is written as `(row, col)`.

Build a **command-line program** that reads commands from an input file and prints the result of each command.

```
<your-run-command> input.txt
```

Keep all data in memory. You don't need a database, UI, or API.

## Commands

| Command | Output |
| --- | --- |
| `create_parking_lot <rows> <cols>` followed by the map lines | `Created a parking lot with <k> usable slots` |
| `park <registration_number> <colour>` | `Allocated slot: (r, c)` or `Sorry, parking lot is full` |
| `leave <row> <col>` | `Slot (r, c) is free` |
| `status` | Occupied slots (see example), or `Parking lot is empty` |
| `registration_numbers_for_cars_with_colour <colour>` | Comma-separated registration numbers, or `Not found` |
| `slot_numbers_for_cars_with_colour <colour>` | Comma-separated slots, or `Not found` |
| `slot_number_for_registration_number <registration_number>` | Slot, or `Not found` |

- `status` and the colour lookups list results in order of **row, then column**.
- If no car is parked, `status` prints `Parking lot is empty` (no table header).
- Colour matching is case-insensitive.
- You may assume the map itself is valid. For any other invalid input, your program must handle it without crashing.

## Sample input ( `input.txt` )

```
create_parking_lot 5 7
E.....P
P#P#P.P
P.....#
PP#P#.P
P####.P
park KA-01-HH-1234 White
park KA-01-HH-9999 White
park KA-01-BB-0001 Black
park KA-01-HH-7777 Red
park KA-01-HH-2701 Blue
park KA-01-HH-3141 Black
park KA-01-HH-5555 Grey
park KA-01-HH-6666 White
park KA-01-HH-8888 Red
park KA-01-HH-1111 Blue
leave 1 7
leave 2 1
status
park KA-01-P-333 White
park DL-12-AA-9999 White
park MH-04-AY-1111 Black
registration_numbers_for_cars_with_colour White
slot_numbers_for_cars_with_colour White
slot_number_for_registration_number KA-01-HH-3141
slot_number_for_registration_number MH-04-AY-1111
slot_number_for_registration_number TN-07-ZZ-0000
```

## Expected output

```
Created a parking lot with 10 usable slots
Allocated slot: (2, 1)
Allocated slot: (2, 3)
Allocated slot: (2, 5)
Allocated slot: (1, 7)
Allocated slot: (2, 7)
Allocated slot: (4, 7)
Allocated slot: (4, 4)
Allocated slot: (5, 7)
Allocated slot: (3, 1)
Allocated slot: (4, 2)
Slot (1, 7) is free
Slot (2, 1) is free
Slot     Registration No    Colour
(2, 3)   KA-01-HH-9999      White
(2, 5)   KA-01-BB-0001      Black
(2, 7)   KA-01-HH-2701      Blue
(3, 1)   KA-01-HH-8888      Red
(4, 2)   KA-01-HH-1111      Blue
(4, 4)   KA-01-HH-5555      Grey
(4, 7)   KA-01-HH-3141      Black
(5, 7)   KA-01-HH-6666      White
Allocated slot: (2, 1)
Allocated slot: (1, 7)
Sorry, parking lot is full
DL-12-AA-9999, KA-01-P-333, KA-01-HH-9999, KA-01-HH-6666
(1, 7), (2, 1), (2, 3), (5, 7)
(4, 7)
Not found
Not found
```

## Performance

The program should handle a **1000 × 1000 grid and 100,000 commands** in a few seconds.

## Submission

Push your code to a **Git repository** (GitHub or GitLab) and share the link. Include:

- Source code
- Unit tests
- A `README.md` with the command to run the program and the tests